

上海科技大学
ShanghaiTech University

Fitting

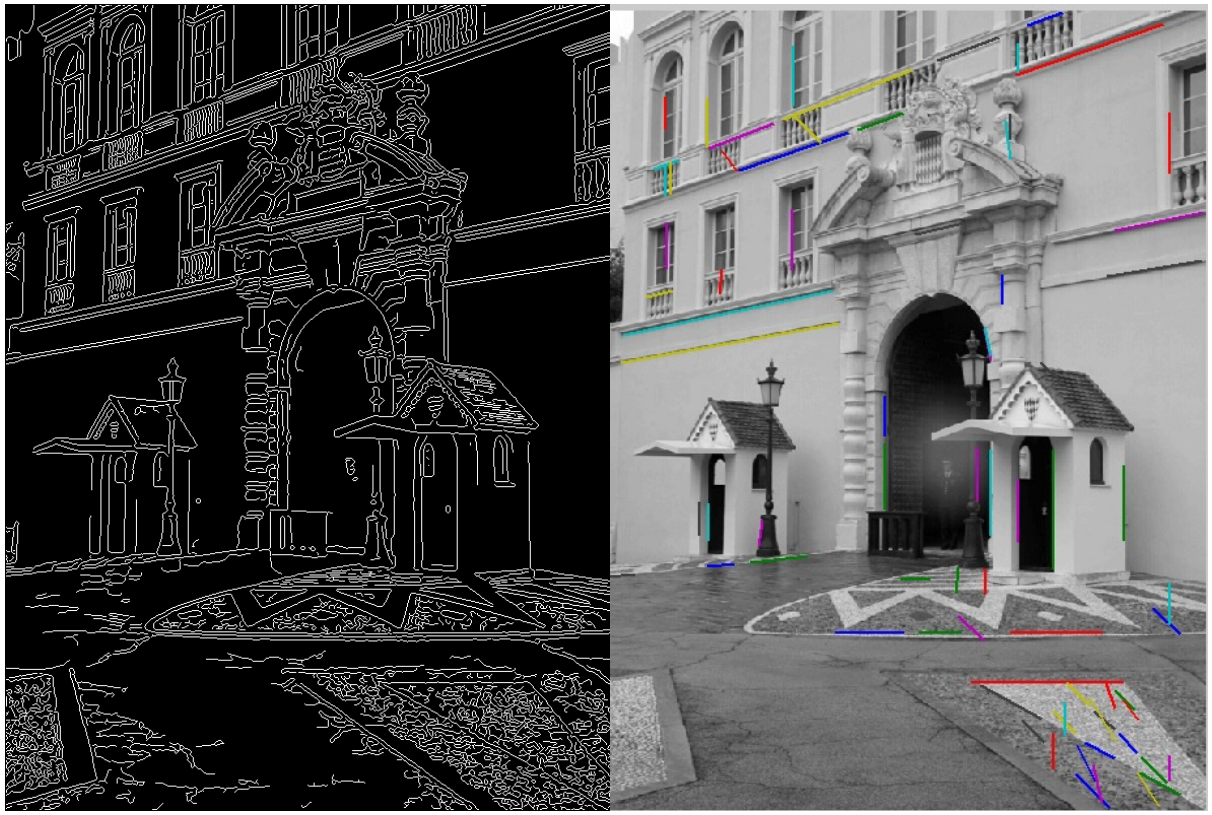
CS172 Computer Vision I

Instructor: Jiayuan Gu

From Features to Models

Feature detection gives us candidate measurements.

Fitting asks whether some of those measurements can be explained by one compact geometric hypothesis.

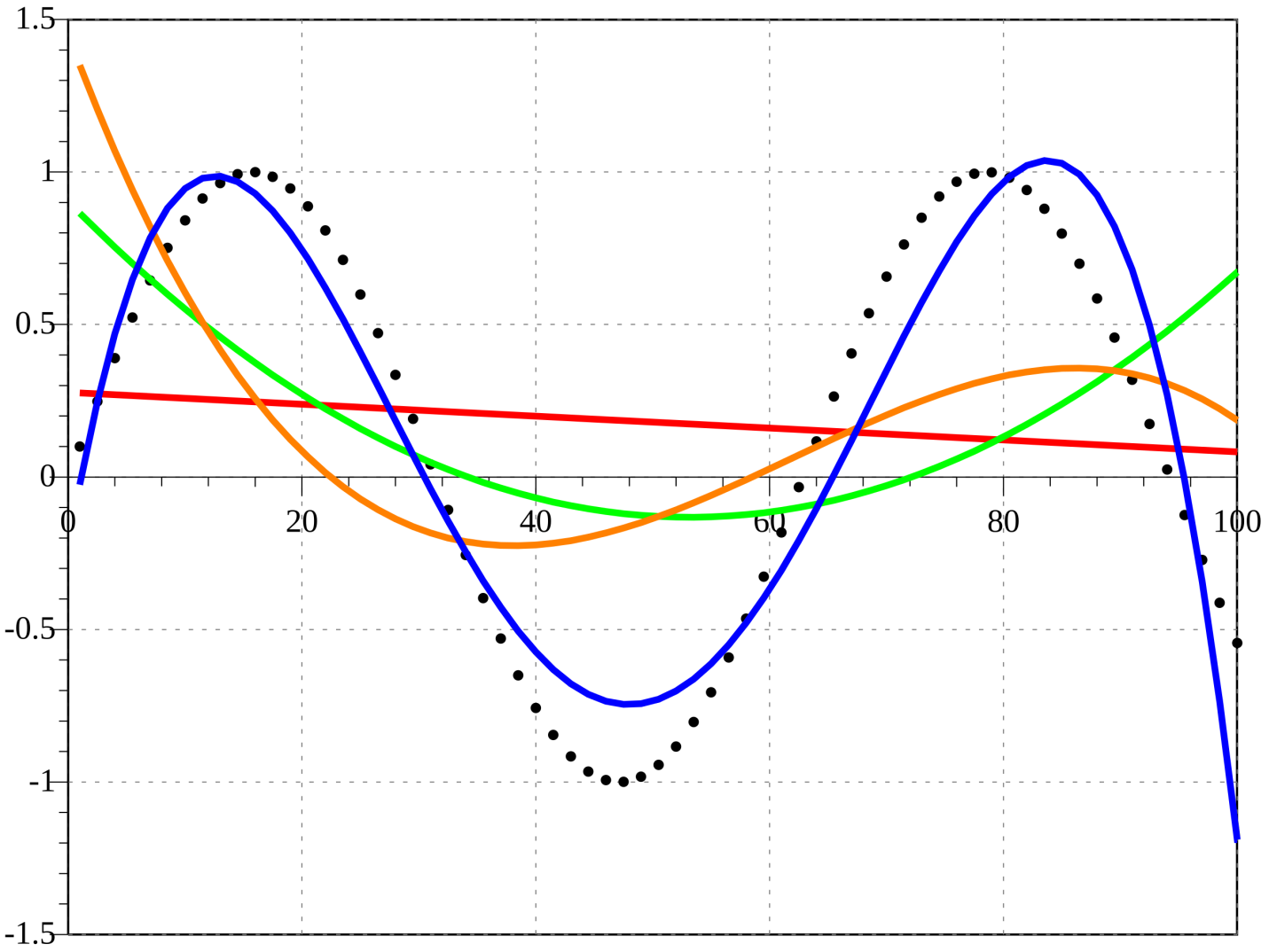


Local evidence: edges, corners, blobs

↓

Geometric structure: line, circle, etc.

Basic Example: Curve Fitting



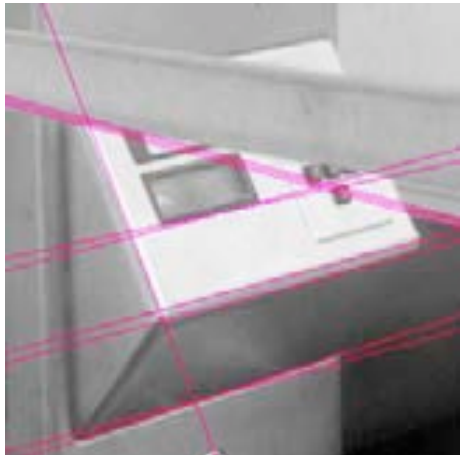
Model: polynomials

“Best fit”: minimal residual

Estimator

From wiki

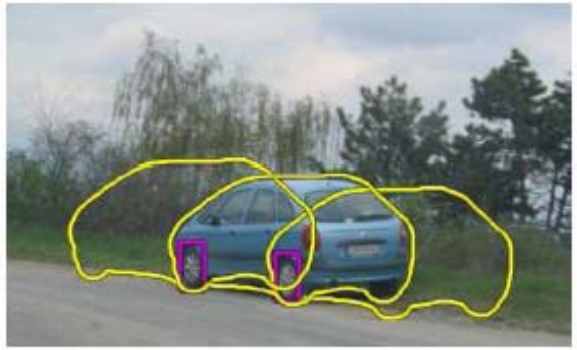
Fitting is a General Framework

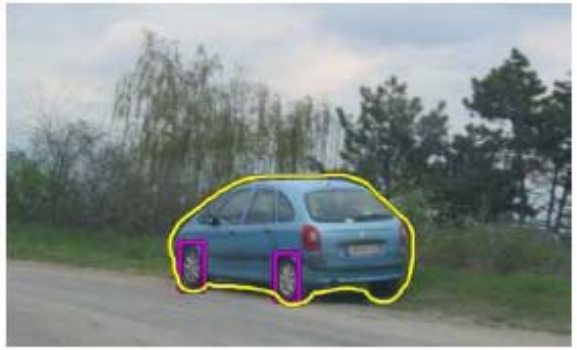


simple model: lines



simple model: circles





complicated model: car

Choose a **parametric model** to represent a set of features

Source: K. Grauman

Application: Line Detection

3D Wireframe detection



(a) Input image (b) 3D wireframe

Learning to Reconstruct 3D Manhattan Wireframes from a Single Image

Lane detection



Image credit: End-to-end Lane Detection

Application: Room Plan



Apple [roomplan]

Real-World Challenges



Case study of line detection

- **Noise** in the detected feature locations
- **Extraneous data:** clutter (outliers)
- **Missing data:** occlusions
- Multiple valid lines

Overview of Fitting Lines

- If we know which points belong to the line, how do we find the “optimal” line parameters?
  - Least squares
- What if there are outliers?
  - Robust fitting, RANSAC
- What if there are many lines?
  - Voting methods: Hough transform (later lecture)
- What if we’re not even sure it’s a line?
  - Model selection (not covered)

Learning Objectives

- **Explain** the role of model fitting in converting low-level features into compact geometric structure, and distinguish **measurement noise** from **outliers**

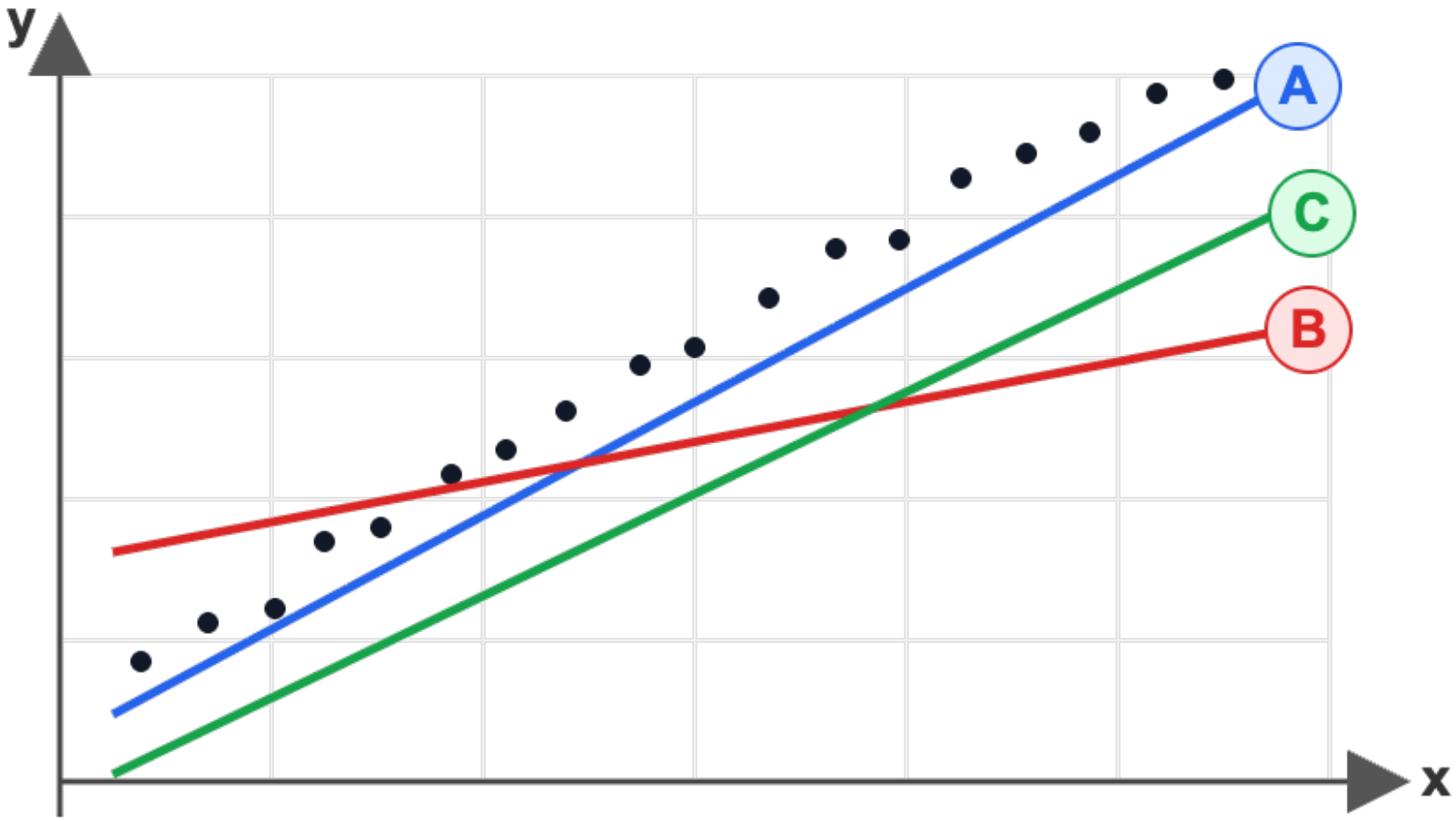
- **Derive** least-squares line fitting in matrix form

- **Compare** least squares, robust estimators, and RANSAC in terms of their assumptions, objective functions, and failure modes

Least Squares

when all measurements are trusted

What does “best fit” mean?



Which line is the “best” one?

Best according to what error?

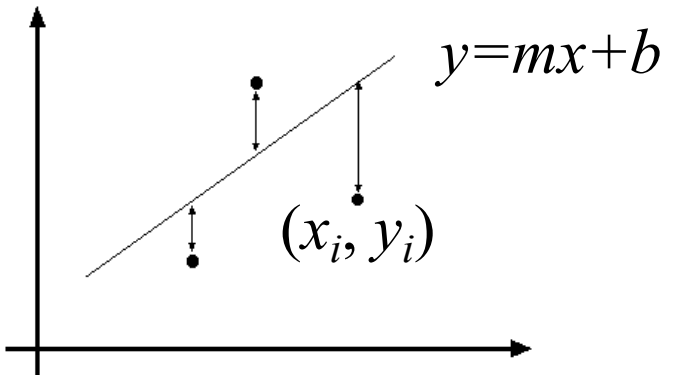
Vertical Residual

Observation: $(x_1, y_1), \ldots, (x_n, y_n)$

Model: $y_i = mx_i + b$

Residual: $E = \sum_{i=1}^{n} (y_i - mx_i - b)^2$

Objective: Find (m, b) to minimize $E(m, b)$



$$E = \|Y - XB\|^2 \quad \text{where} \quad Y = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix} \qquad X = \begin{bmatrix} x_1 & 1 \\ \vdots & \vdots \\ x_n & 1 \end{bmatrix} \qquad B = \begin{bmatrix} m \\ b \end{bmatrix}$$

Matrix form

Least Squares Solution

$$E = \|Y - XB\|^2 \quad \text{where} \quad Y = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix} \qquad X = \begin{bmatrix} x_1 & 1 \\ \vdots & \vdots \\ x_n & 1 \end{bmatrix} \qquad B = \begin{bmatrix} m \\ b \end{bmatrix}$$

$$E = \|Y - XB\|^2 = (Y - XB)^T (Y - XB) = Y^T Y - 2(XB)^T Y + (XB)^T (XB)$$

$$\frac{dE}{dB} = 2X^T XB - 2X^T Y = 0$$

http://faculty.bicmr.pku.edu.cn/~wenzw/bigdata/matrix-cook-book.pdf sec 2.4

$$\boxed{X^T XB = X^T Y}$$

Normal equations: least squares solution to $XB=Y$

Normal Equation

Given a matrix equation

$$A\,\mathbf{x} = \mathbf{b},$$

the normal equation is that which minimizes the sum of the square differences between the left and right sides:

$$A^T A\,\mathbf{x} = A^T\,\mathbf{b}.$$

It is called a normal equation because $\mathbf{b} - A\,\mathbf{x}$ is normal to the range of A.

Here, $A^T A$ is a normal matrix.

Implementation trick: Call np.linalg.lstsq(X, y, rcond=None)[0] instead of computing the inverse

Total Least Squares

A “better” definition of residual

Problem with “vertical” least squares

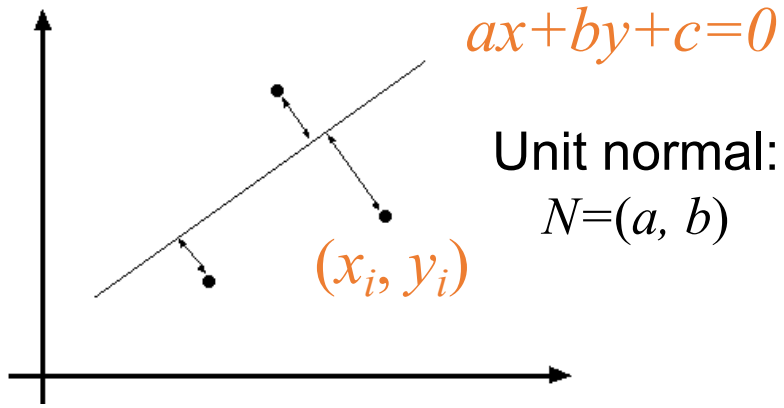
- Consider the following case: if the same data points are rotated by 90 degrees, *should the quality of the fitted geometric line change just because the coordinate system is rotated*?

- The “vertical” least squares are **not rotation-invariant**

- Besides, does the line equation of $y_{\mathrm{i}} = mx_i + b$ work for all lines?

- The ordinary line equation becomes problematic near a vertical line

Total Least Squares



Distance between point (x_i, y_i) and line $ax+by+c=0$ $(a^2+b^2=1)$: $|ax_i + by_i + c|$

Find (a, b, c) to minimize the sum of squared perpendicular distances

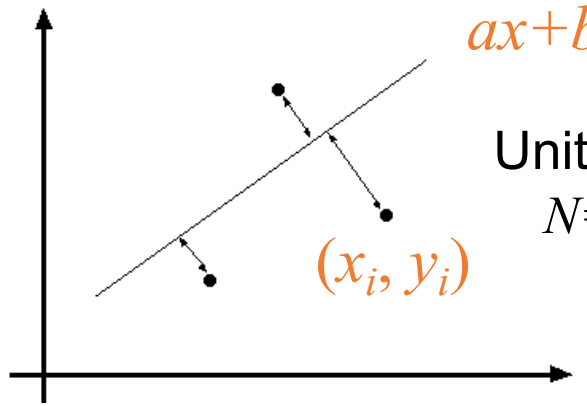
$$E = \sum_{i=1}^{n} (ax_i + by_i + c)^2$$

Modified from S. Lazebnik

Total Least Squares

Find (a, b, c) to minimize the sum of squared perpendicular distances

$$E = \sum_{i=1}^{n} (ax_i + by_i + c)^2$$



$$\frac{\partial E}{\partial c} = \sum_{i=1}^{n} 2(ax_i + by_i + c) = 0$$

$$c = -\frac{a}{n}\sum_{i=1}^{n} x_i - \frac{b}{n}\sum_{i=1}^{n} y_i = -a\bar{x} - b\bar{y}$$

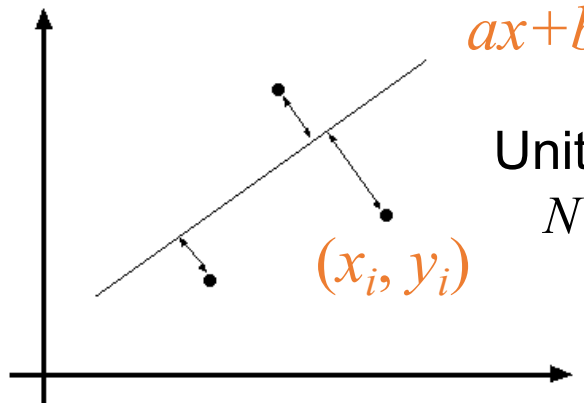
$$E = \sum_{i=1}^{n} (a(x_i - \bar{x}) + b(y_i - \bar{y}))^2 = \left\| \begin{bmatrix} x_1 - \bar{x} & y_1 - \bar{y} \\ \vdots & \vdots \\ x_n - \bar{x} & y_n - \bar{y} \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} \right\|^2 = (UN)^T (UN)$$

Modified from S. Lazebnik

Total Least Squares

Find (a, b, c) to minimize the sum of squared perpendicular distances

$$E = \sum_{i=1}^{n} (ax_i + by_i + c)^2$$



$$E = \sum_{i=1}^{n} (a(x_i - \bar{x}) + b(y_i - \bar{y}))^2 = \left\| \begin{bmatrix} x_1 - \bar{x} & y_1 - \bar{y} \\ \vdots & \vdots \\ x_n - \bar{x} & y_n - \bar{y} \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} \right\|^2 = (UN)^T (UN)$$

$$\frac{dE}{dN} = 2(U^T U)N = 0$$

Solution to $(U^T U)N = 0$, subject to $\|N\|^2 = 1$:
eigenvector of $U^T U$ associated with the smallest eigenvalue
(least squares solution to *homogeneous linear system* $UN = 0$)

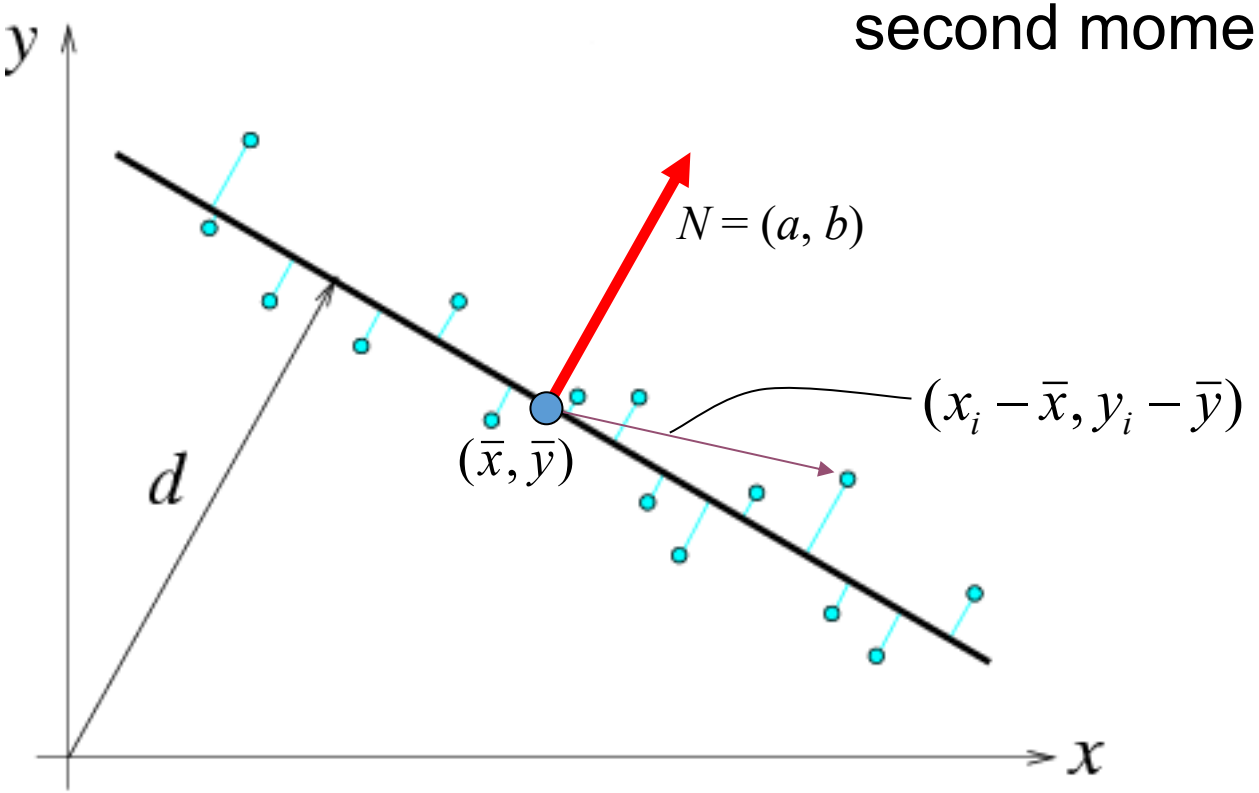
Modified from S. Lazebnik

Total Least Squares

$$U = \begin{bmatrix} x_1 - \bar{x} & y_1 - \bar{y} \\ \vdots & \vdots \\ x_n - \bar{x} & y_n - \bar{y} \end{bmatrix}$$

$$U^T U = \begin{bmatrix} \sum_{i=1}^{n} (x_i - \bar{x})^2 & \sum_{i=1}^{n} (x_i - \bar{x})(y_i - \bar{y}) \\ \sum_{i=1}^{n} (x_i - \bar{x})(y_i - \bar{y}) & \sum_{i=1}^{n} (y_i - \bar{y})^2 \end{bmatrix}$$

second moment matrix

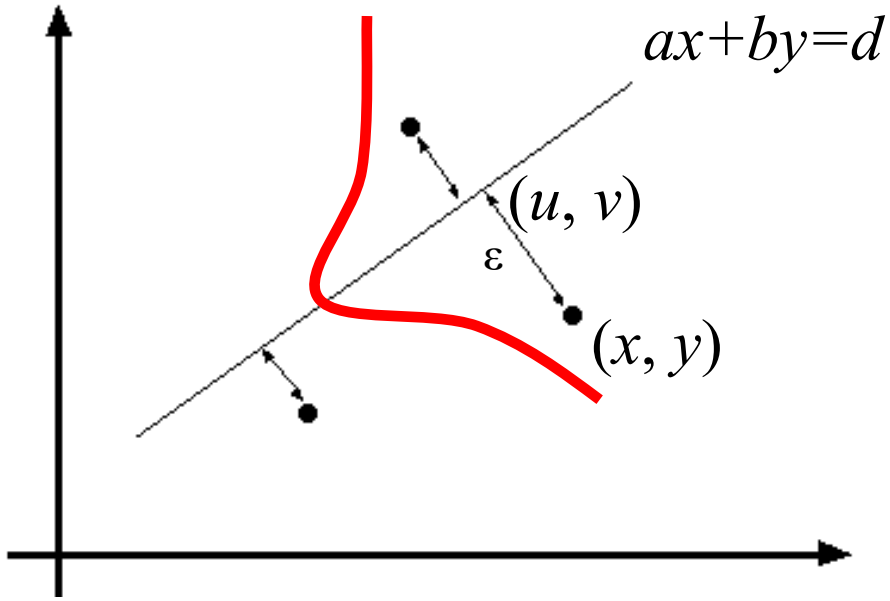


F&P (2nd ed.) sec. 22.1

Least squares as likelihood maximization

Generative model: points are sampled independently and corrupted by Gaussian noise in the direction perpendicular to the line

$$\begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} u \\ v \end{pmatrix} + \varepsilon \begin{pmatrix} a \\ b \end{pmatrix}$$



Likelihood of points given line parameters (*a*, *b*, *d*):

$$P(x_1, y_1, \ldots, x_n, y_n \mid a, b, d) = \prod_{i=1}^{n} P(x_i, y_i \mid a, b, d) \propto \prod_{i=1}^{n} \exp\left(-\frac{(ax_i + by_i - d)^2}{2\sigma^2} \right)$$

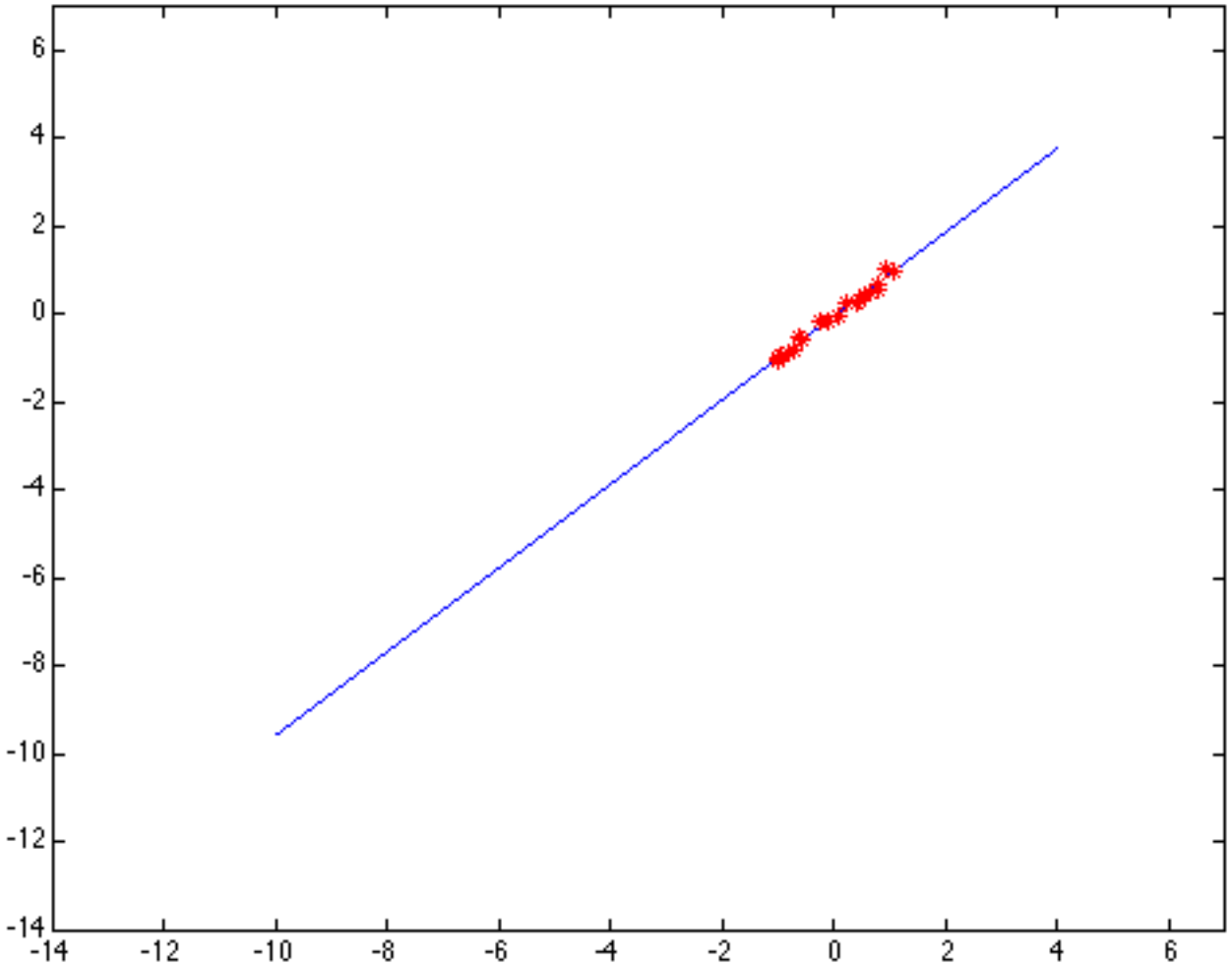
Log-likelihood: $L(x_1, y_1, \ldots, x_n, y_n \mid a, b, d) = -\frac{1}{2\sigma^2} \sum_{i=1}^{n} (ax_i + by_i - d)^2$

Modified from S. Lazebnik

Outliers and Robust Fitting

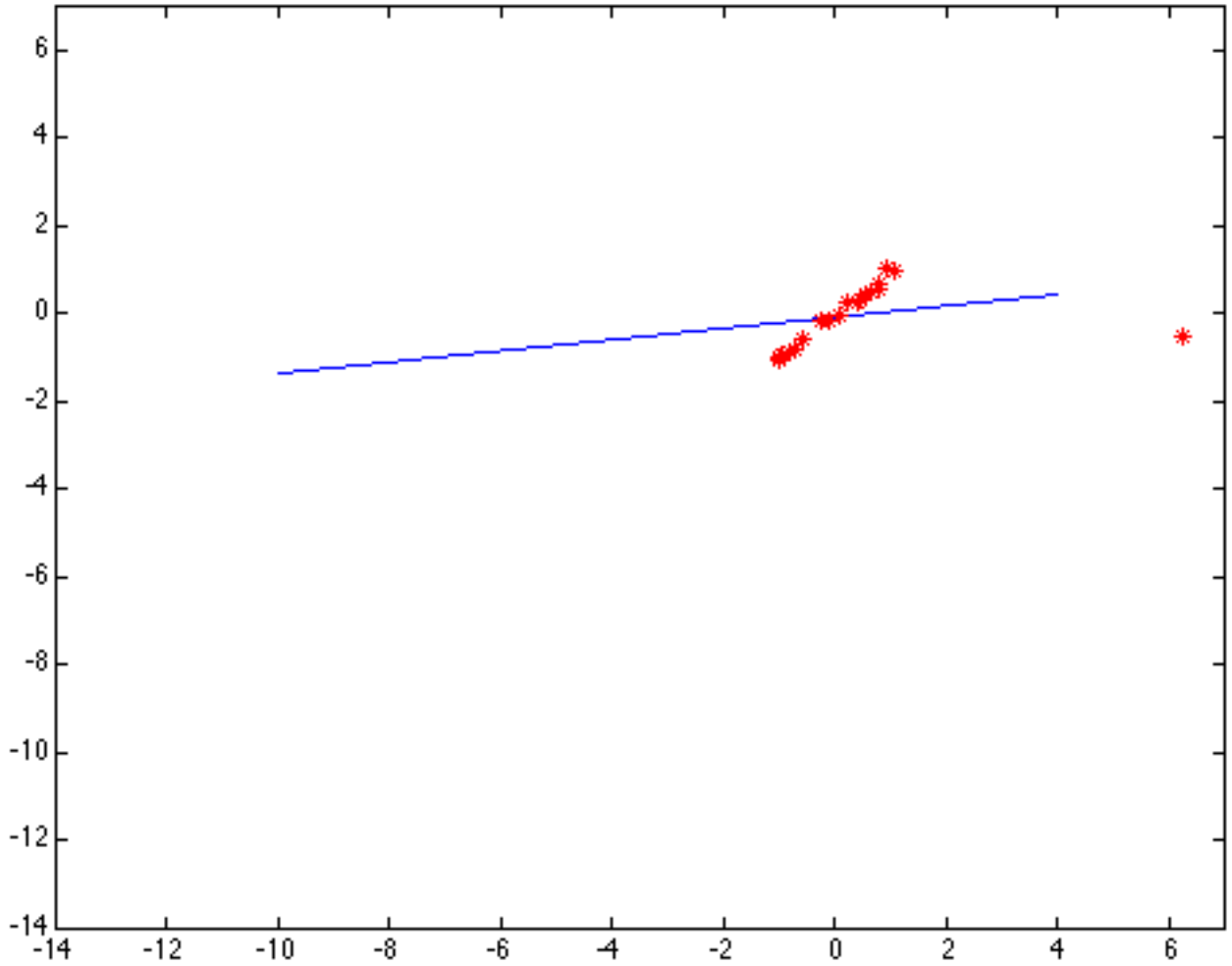
Robustness to Noise

- Least squares fit to the red points:



Least squares: Robustness to noise

- Least squares fit with an outlier:

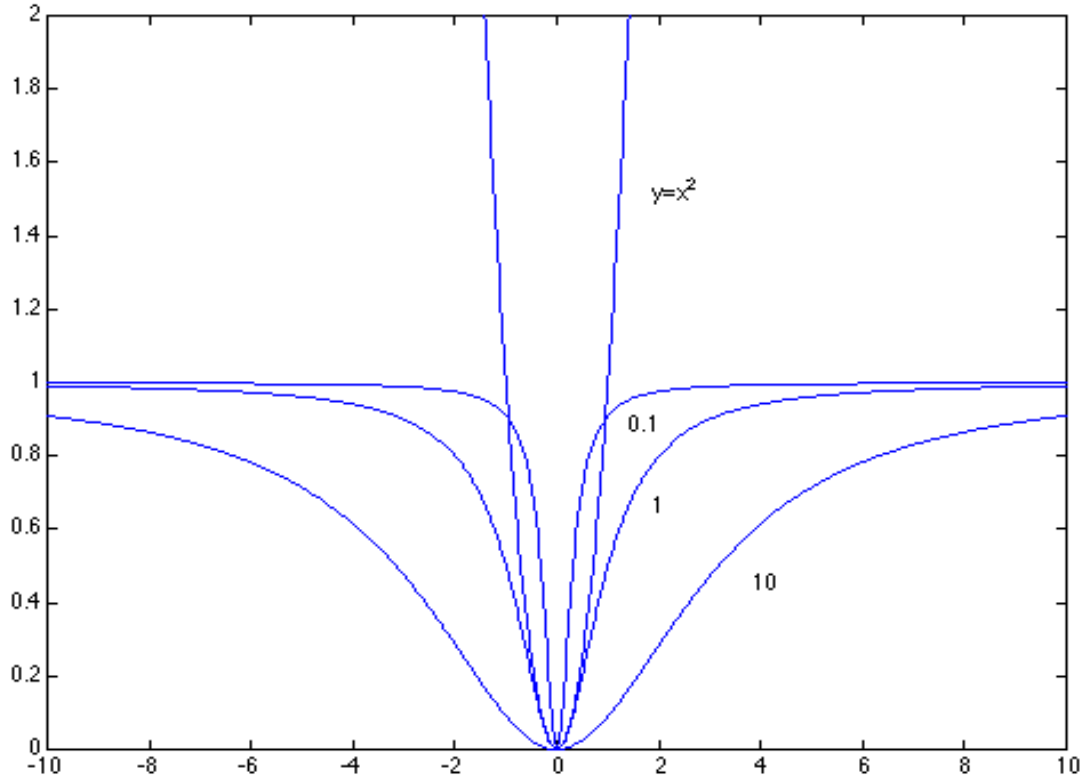


Problem: squared error heavily penalizes outliers

Robust Estimators

General approach: find model parameters θ that minimize $\sum_i \rho\left(r_i\left(x_i, \theta\right); \sigma\right)$

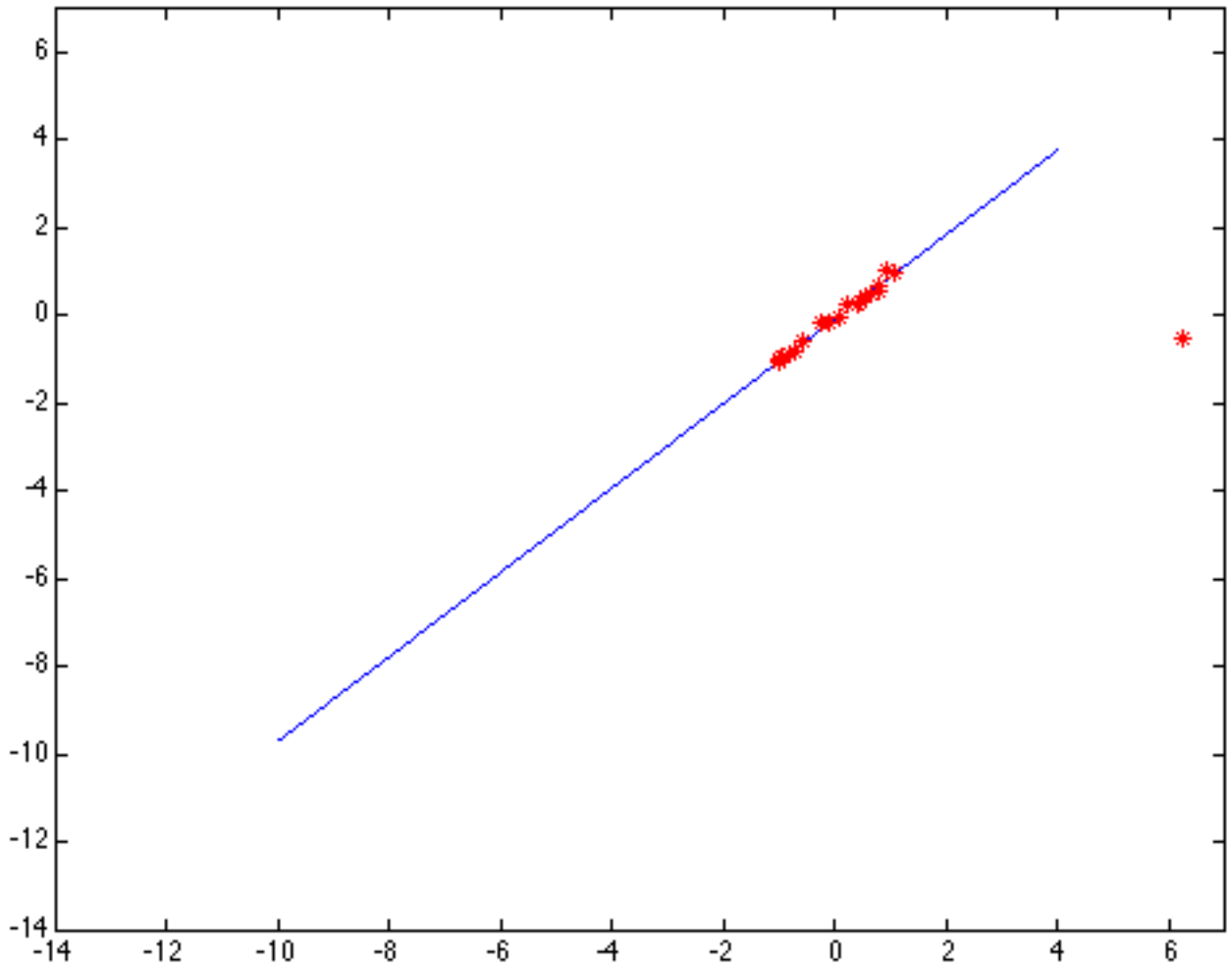
$r_i(x_i, \theta)$ – residual of i-th point w.r.t. model parameters θ
ρ – robust function with scale parameter σ



$$\rho(u;\sigma) = \frac{u^2}{\sigma^2 + u^2}$$

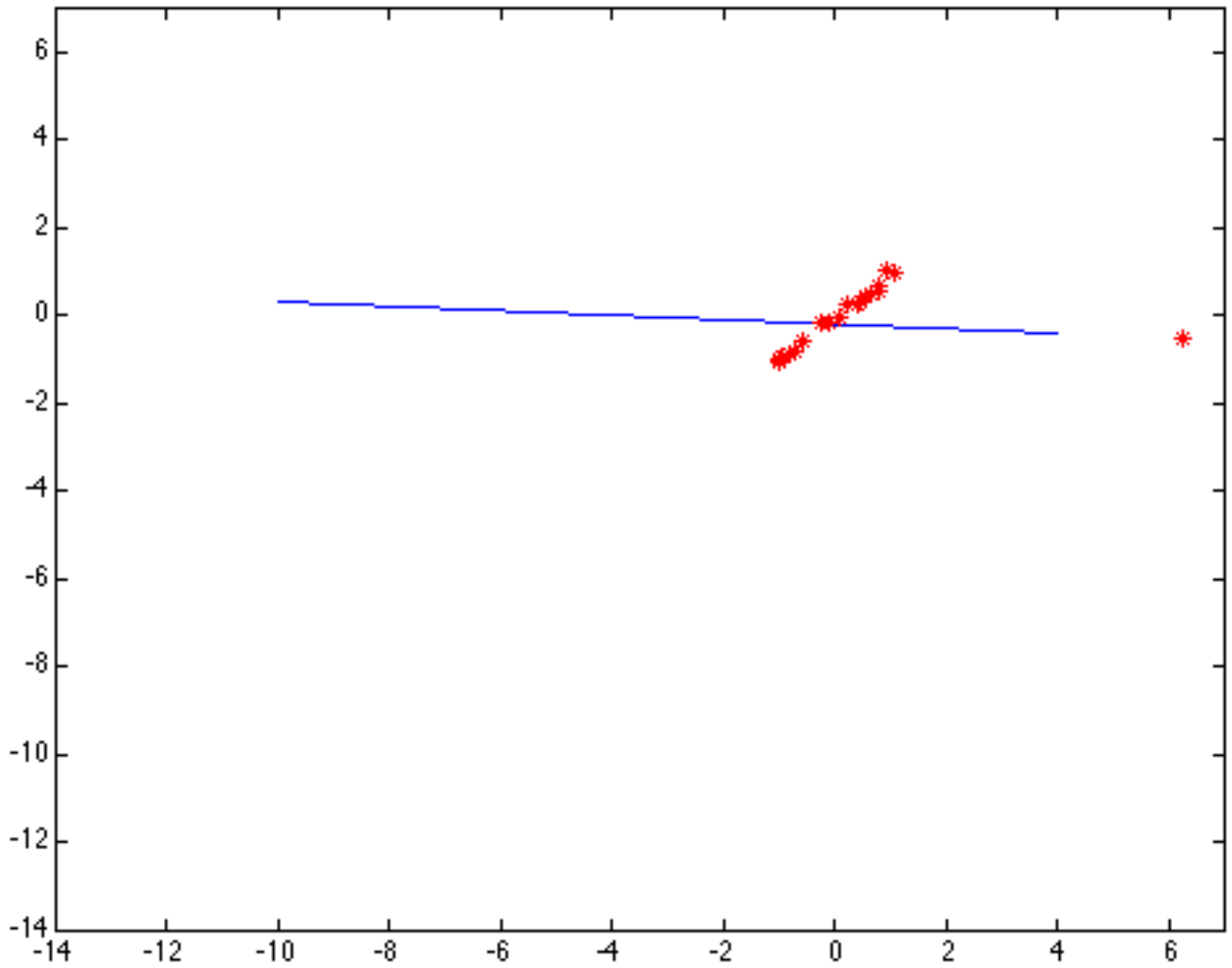
The robust function ρ behaves like squared distance for small values of the residual *u* but saturates for larger values of *u*

Choosing the scale: just right



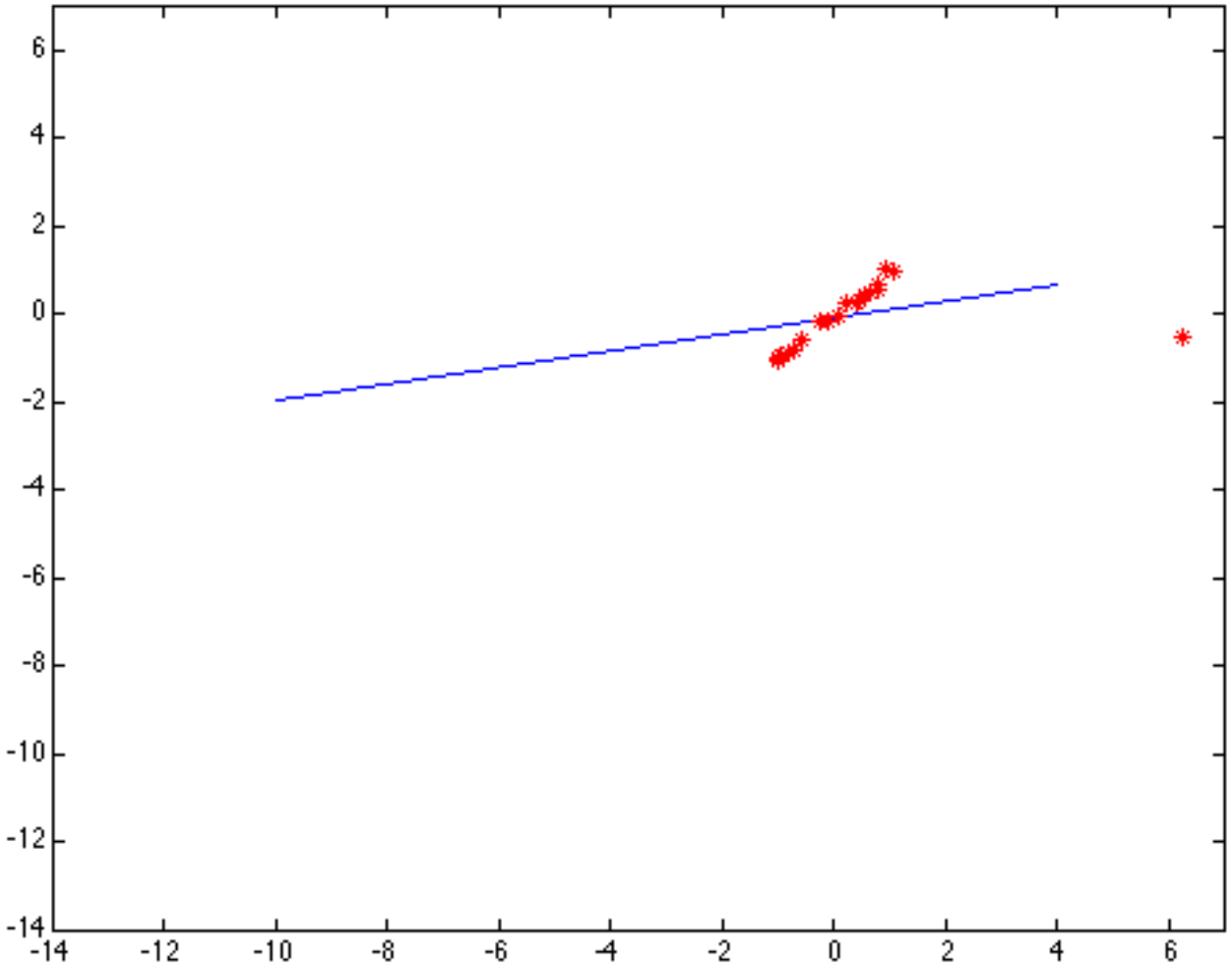
The effect of the outlier is minimized

Choosing the scale: too small



The error value is almost the same for every point and the fit is very poor

Choosing the scale: too large



Behaves much the same as least squares

Robust Estimation Summary

- Robust fitting is a nonlinear optimization problem that must be solved iteratively
- Least squares solution can be used for initialization
- Scale of robust function should be chosen adaptively based on median residual

RANSAC

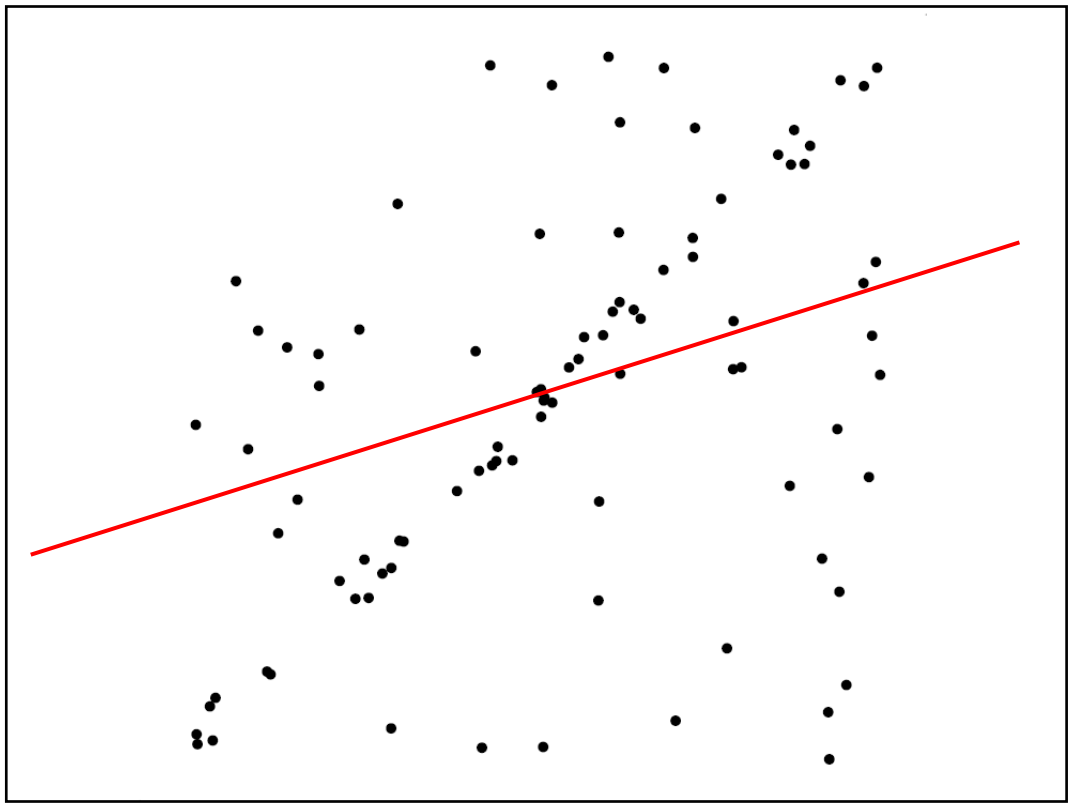
fitting when there may be many outliers

RANSAC

- Robust fitting can deal with a few outliers – what if we have very many?
- Random sample consensus (RANSAC):
  Very general framework for model fitting in the presence of outliers
- Outline
  - Choose a small subset of points uniformly at random
  - Fit a model to that subset
  - Find all remaining points that are “close” to the model and reject the rest as outliers
  - Do this many times and choose the best model

M. A. Fischler, R. C. Bolles. Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography. Comm. of the ACM, Vol 24, pp 381-395, 1981.

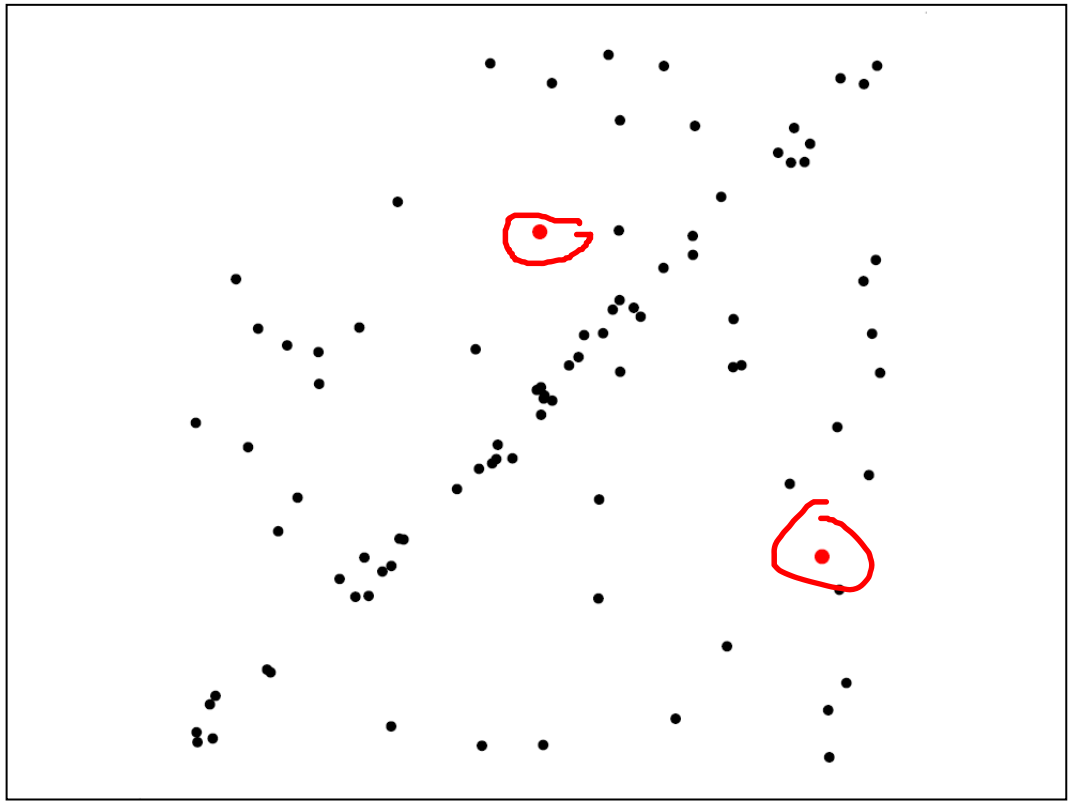
RANSAC for Line Fitting



Least Squares

Source: R. Raguram

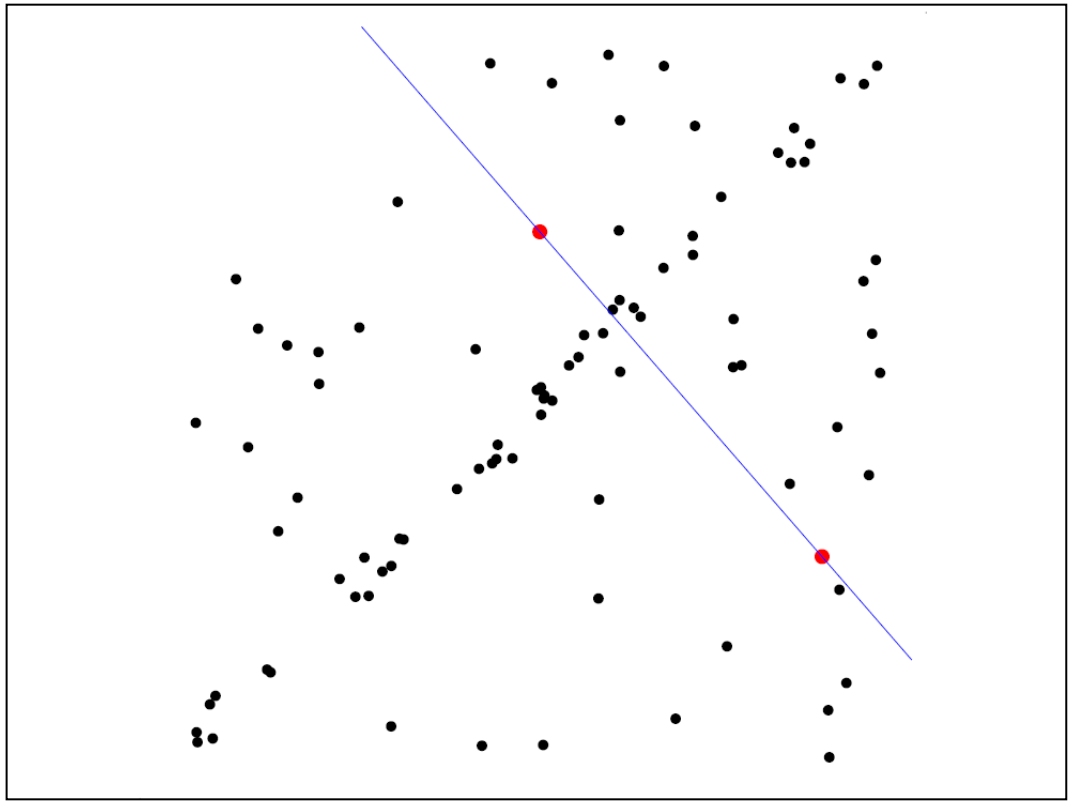
RANSAC for Line Fitting



Step 1: Randomly select minimal subset of points

Source: R. Raguram

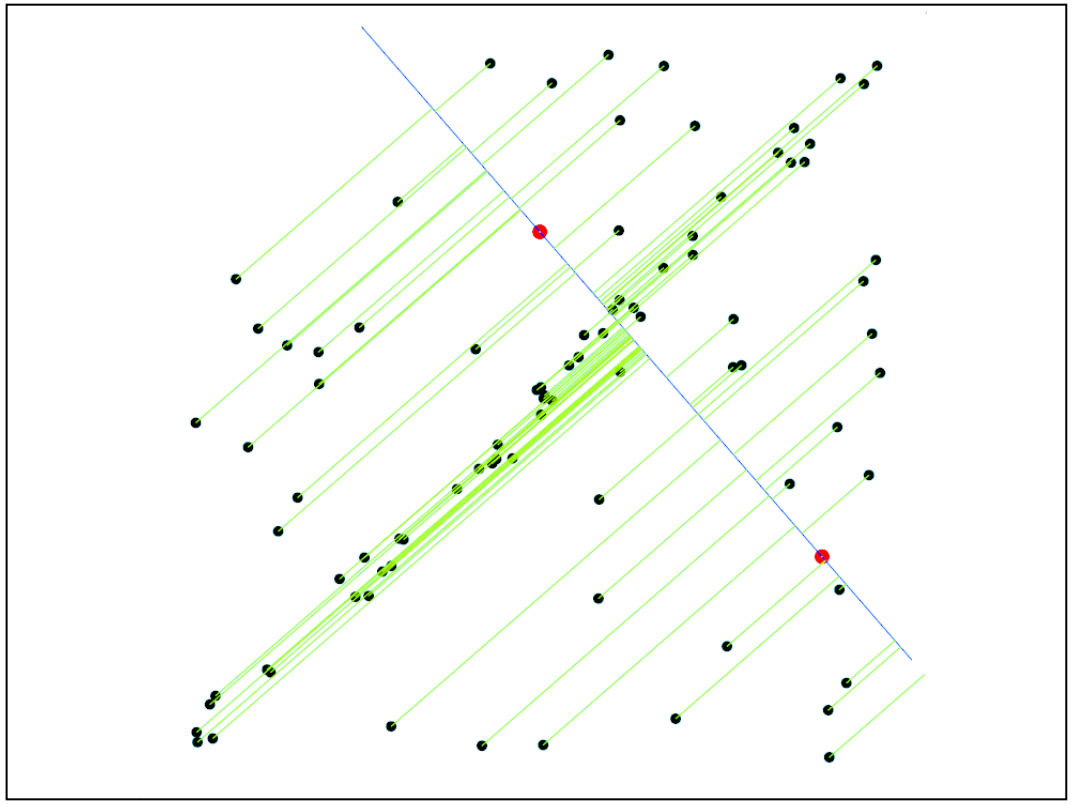
RANSAC for Line Fitting



Step 1: Randomly select minimal subset of points
Step 2: Hypothesize a model

Source: R. Raguram

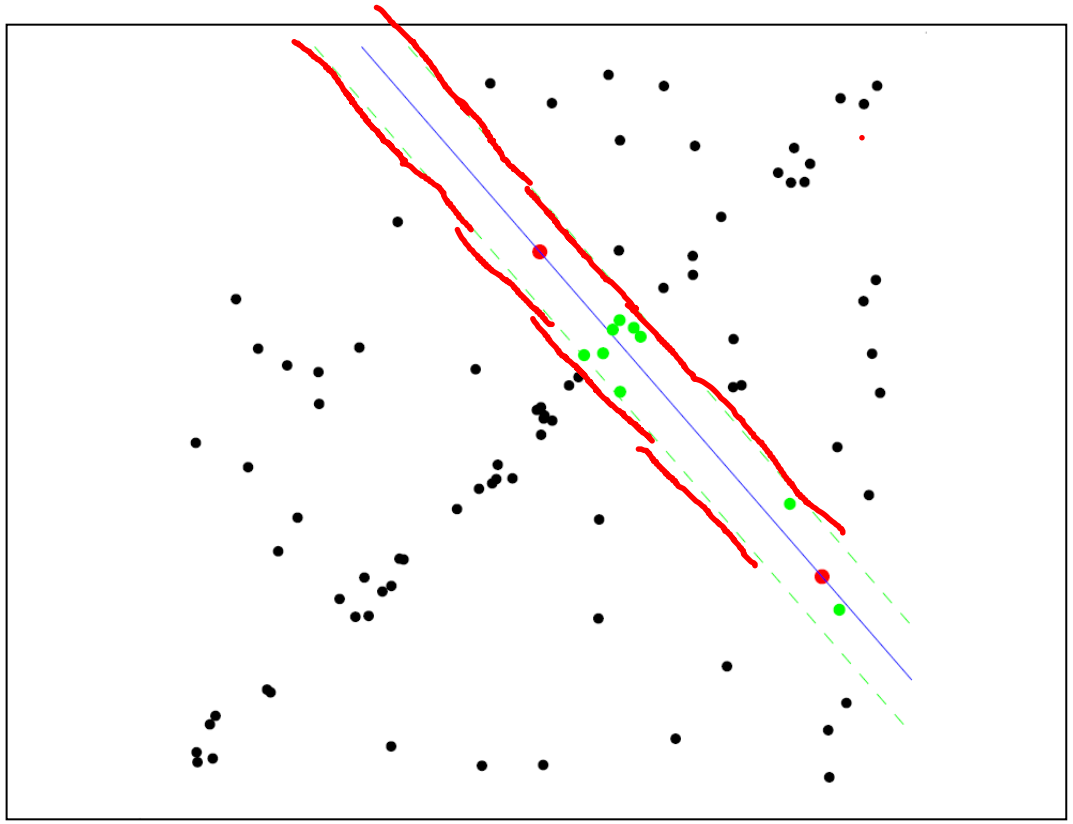
RANSAC for Line Fitting



Step 1: Randomly select minimal subset of points
Step 2: Hypothesize a model
Step 3: Compute error function

Source: R. Raguram

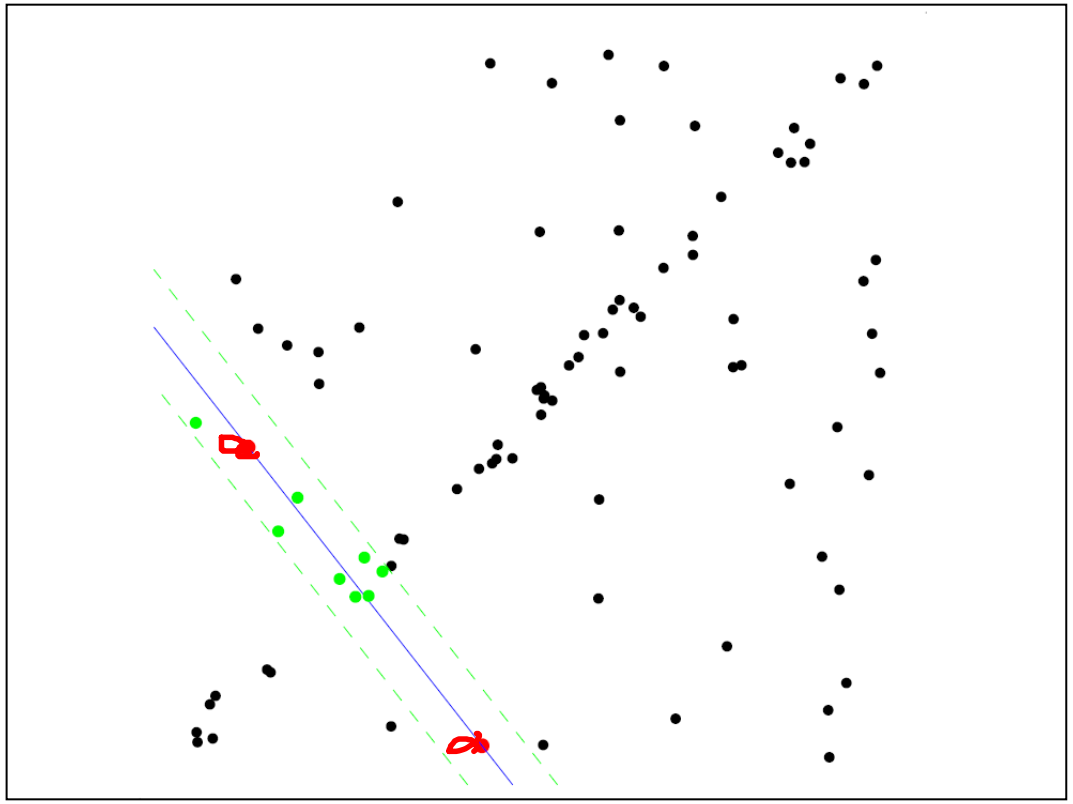
RANSAC for Line Fitting



Step 1: Randomly select minimal subset of points
Step 2: Hypothesize a model
Step 3: Compute error function
Step 4: Select points consistent with model

Source: R. Raguram

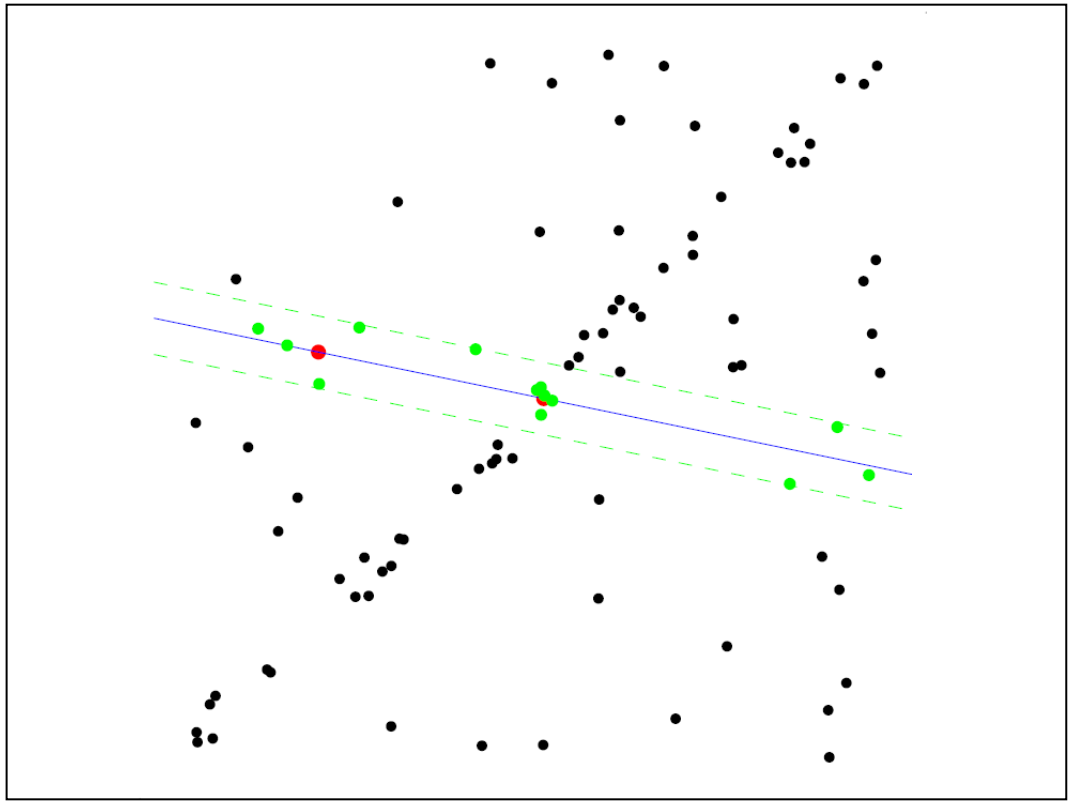
RANSAC for Line Fitting



Step 1: Randomly select minimal subset of points
Step 2: Hypothesize a model
Step 3: Compute error function
Step 4: Select points consistent with model
Step 5: Repeat *hypothesize-and-verify* loop

Source: R. Raguram

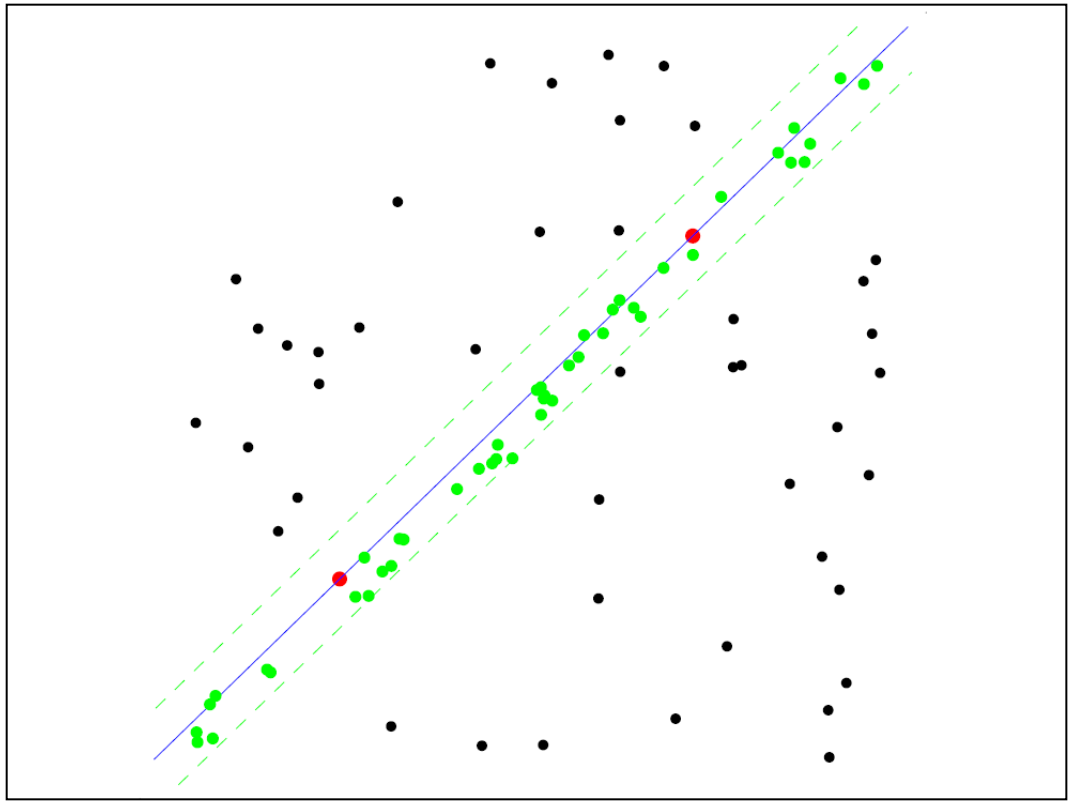
RANSAC for Line Fitting



Step 1: Randomly select minimal subset of points
Step 2: Hypothesize a model
Step 3: Compute error function
Step 4: Select points consistent with model
Step 5: Repeat *hypothesize-and-verify* loop

Source: R. Raguram

RANSAC for Line Fitting

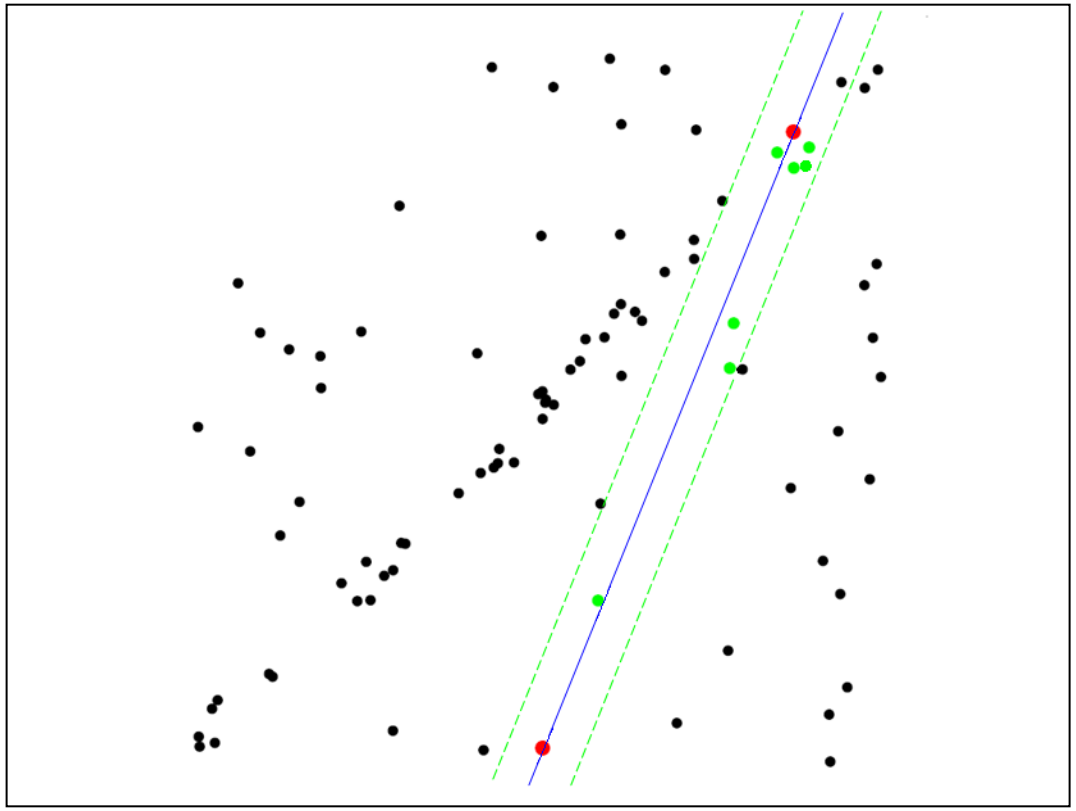


Uncontaminated samples

Step 1: Randomly select minimal subset of points
Step 2: Hypothesize a model
Step 3: Compute error function
Step 4: Select points consistent with model
Step 5: Repeat *hypothesize-and-verify* loop

Source: R. Raguram

RANSAC for Line Fitting



Continue to search until certain condition is satisfied

Step 1: Randomly select minimal subset of points
Step 2: Hypothesize a model
Step 3: Compute error function
Step 4: Select points consistent with model
Step 5: Repeat *hypothesize-and-verify* loop

Source: R. Raguram

RANSAC for Line Fitting

- Repeat ***N*** times:
- Draw ***s*** points uniformly at random
- Fit line to these ***s*** points
- Find *inliers* to this line among the remaining points (i.e., points whose distance from the line is less than ***t***)
- If there are ***d*** or more inliers, accept the line and refit using all inliers

Choosing the parameters

- Initial number of points ***s***
  - Typically minimum number needed to fit the model

- Distance threshold ***t***
  - Choose ***t*** so probability for inlier is p (e.g. 0.95)
  - Zero-mean Gaussian noise with standard deviation σ
  - $t^2=3.84\sigma^2$ follows χ^2 distribution for 2D line (empirically)

- Number of samples ***N***
  - Choose ***N*** so that, with probability p, at least one random sample is free from outliers (e.g. $p=0.99$). Denote outlier ratio: e

Source: M. Pollefeys

Choosing the parameters

- Initial number of points ***s***
- Distance threshold ***t***
- Number of samples ***N***
  - Choose ***N*** so that, with probability p, at least one random sample is free from outliers (e.g. p=0.99). Denote outlier ratio: e

$$\left(1-\left(1-e\right)^{s}\right)^{N}=1-p$$

$$N=\log\left(1-p\right)/\log\left(1-\left(1-e\right)^{s}\right)$$

	proportion of outliers e						
s	5%	10%	20%	25%	30%	40%	50%
2	2	3	5	6	7	11	17
3	3	4	7	9	11	19	35
4	3	5	9	13	17	34	72
5	4	6	12	17	26	57	146
6	4	7	16	24	37	97	293
7	4	8	20	33	54	163	588
8	5	9	26	44	78	272	1177

Source: M. Pollefeys

Choosing the parameters

- Initial number of points ***s***
- Distance threshold ***t***
- Number of samples ***N***
  - Denote outlier ratio: e
- Consensus set size ***d***
  - Should match expected inlier ratio $(1\text{-}e)$

Source: M. Pollefeys

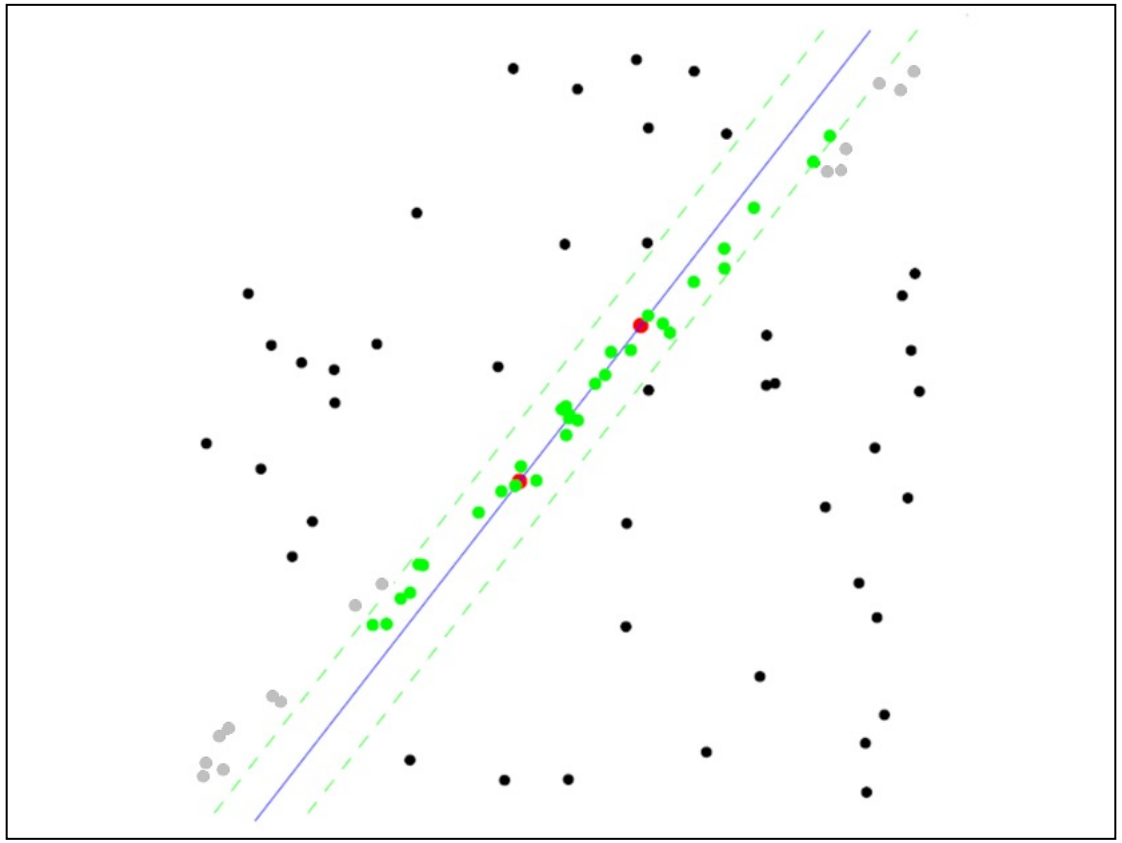
Adaptively determining the number of samples

- The outlier ratio e is often unknown a priori
- Thus, starting from the worst case, e.g. 50%, and adapt if more inliers are found
- Adaptive procedure:
  - Start from $N=\infty$, $sample_count = 0$
  - While $N > sample_count$
    - Choose a sample and count the number of inliers
    - If the inlier ratio is the highest of any found so far, set $e = 1 - (number\ of\ inliers)/(total\ number\ of\ points)$
    - Re-compute N from e: $N = \log(1-p)/\log\left(1-(1-e)^s\right)$
    - Increment the $sample_count$ by 1

Source: M. Pollefeys

RANSAC pros and cons

- Pros
  - Simple and general
  - Applicable to many different problems
  - Often works well in practice
- Cons
  - Lots of parameters to tune
  - Doesn’t work well for low inlier ratios (too many iterations, or can fail completely)
  - Can’t always get a good initialization of the model based on the minimum number of samples

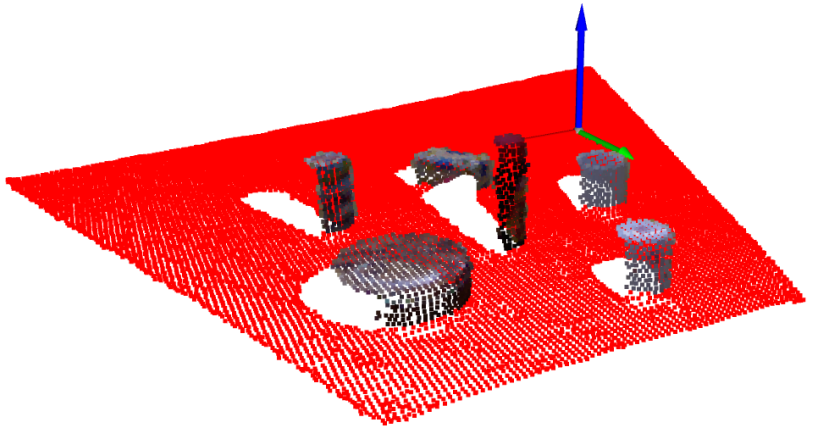


Summary

- ✓ If we know which points belong to the line, how do we find the "optimal" line parameters?
  - ✓Least squares
- ✓ What if there are outliers?
  - ✓Robust fitting, RANSAC
- • What if there are many lines?

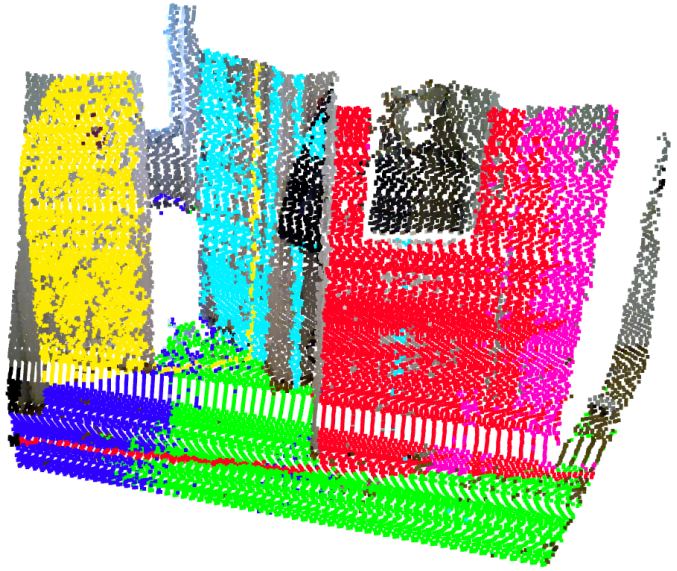
Exercise: Plane Fitting





Model: 3D plane
How many points are needed?
Residual?
Least square solutions?





Model: $ax + by + cz + d = 0$
3 points are needed
Residual: $r = \frac{|ax+by+cz+d|}{\sqrt{a^2+b^2+c^2}}$

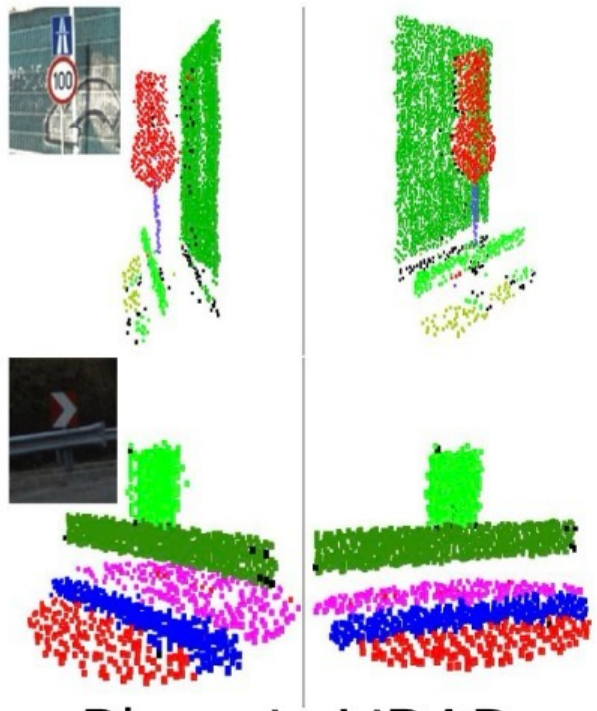
Figures from https://github.com/salykova/ransac

Exploiting the Spatial Coherence of Geometric Data

Motivation: In vision, we usually have geometric data, e.g., 3D points, where the points often originate from spatially coherent structures.



Two-view geometry. (*Left*) Rigid motions in two views. (*Right*) 1st images of image pairs with the inliers of homographies.



Planes in LiDAR data.



Vanishing points (similar line directions)

Reading List

- **Szeliski, *Computer Vision: Algorithms and Applications*, 2e**
  - § 4.1.3 **Robust Data Fitting**; § 8.1.1 **2D Alignment Using Least Squares**; § 8.1.4 **Robust Least Squares and RANSAC**
- **Forsyth & Ponce, *Computer Vision: A Modern Approach*, 2e**
  - Ch. 22, especially § 22.1 **Linear Least Squares** and § 22.1.4 **Fitting a Line to Points in a Plane**
- **Fischler, M. A. & Bolles, R. C. (1981).** *"Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography."*
- Scipy's implementation of least squares